Particle swarm optimization

Code:

```
import random
import math

# Engineering constants
P = 6000
L = 14
E = 30 * 10**6
G = 12 * 10**6

# Objective function: manufacturing cost
def objective(x):
    h, l, t, b = x
    return 1.10471 * h**2 * l + 0.04811 * t * b * (14 + l)

# Calculate engineering constraints
def constraints(x):
    h, l, t, b = x

    tau1 = P / (math.sqrt(2) * h * l)

    M = P * (L + l / 2)

    R = math.sqrt(l**2 / 4 + ((h + t) / 2)**2)
```

```
J = 2 * math.sqrt(2) * h * l * (
    l**2 / 12 + ((h + t) / 2)**2
)

tau2 = M * R / J

tau = math.sqrt(
    tau1**2 + 2 * tau1 * tau2 * l / (2 * R) + tau2**2
)

sigma = (6 * P * L) / (b * t**2)

delta = (4 * P * L**3) / (E * t**3 * b)

Pc = (
    4.013 * E * math.sqrt(t**2 * b**6 / 36) / L**2
) * (
    1 - (t / (2 * L)) * math.sqrt(E / (4 * G))
)

# All constraints must be <= 0
g = [
    (tau - 13600) / 13600,
    (sigma - 30000) / 30000,
    (h - b) / 2,
    (0.10471 * h**2 + 0.04811 * t * b * (14 + l) - 5) / 5,
    (0.125 - h) / 0.125,
    (delta - 0.25) / 0.25,
```

```
        (P - Pc) / P
    ]

    return g, tau, sigma, delta, Pc

# Fitness function using penalty method
def fitness(x):
    cost = objective(x)
    g, _, _, _, _ = constraints(x)

    penalty = 0

    for value in g:
        if value > 0:
            penalty += value**2

    return cost + 10000000 * penalty

# Particle Swarm Optimization
def pso():

    # Read PSO parameters
    n = int(input("Number of particles: "))
    iterations = int(input("Maximum iterations: "))
    seed = int(input("Random seed: "))

    random.seed(seed)

    # Variables: h, l, t, b
```

```
lower = [0.125, 0.1, 0.1, 0.1]
upper = [2.0, 10.0, 10.0, 2.0]

dimensions = 4

# PSO parameters
c1 = 1.49445
c2 = 1.49445

# Initialize particle positions and velocities
positions = []
velocities = []

for i in range(n):
    position = []
    velocity = []

    for d in range(dimensions):
        position.append(
            random.uniform(lower[d], upper[d])
        )

        velocity.append(
            random.uniform(
                -0.1 * (upper[d] - lower[d]),
                0.1 * (upper[d] - lower[d])
            )
        )
```

```
    positions.append(position)
    velocities.append(velocity)

# Initialize personal best
pbest = [p[:] for p in positions]
pbest_fitness = [fitness(p) for p in positions]

# Initialize global best
best_index = pbest_fitness.index(min(pbest_fitness))
gbest = pbest[best_index][:]
gbest_fitness = pbest_fitness[best_index]

# Main PSO loop
for iteration in range(iterations):

    # Decrease inertia weight gradually
    w = 0.9 - 0.5 * iteration / iterations

    for i in range(n):

        for d in range(dimensions):

            r1 = random.random()
            r2 = random.random()

            # Update velocity
            velocities[i][d] = (
                w * velocities[i][d]
                + c1 * r1 * (pbest[i][d] - positions[i][d])
```

```
            + c2 * r2 * (gbest[d] - positions[i][d])
        )

        # Limit velocity
        vmax = 0.2 * (upper[d] - lower[d])

        velocities[i][d] = max(
            -vmax,
            min(vmax, velocities[i][d])
        )

        # Update position
        positions[i][d] += velocities[i][d]

        # Keep position within limits
        positions[i][d] = max(
            lower[d],
            min(upper[d], positions[i][d])
        )

    # Calculate new fitness
    current_fitness = fitness(positions[i])

    # Update personal best
    if current_fitness < pbest_fitness[i]:

        pbest[i] = positions[i][:]
        pbest_fitness[i] = current_fitness
```

```
        # Update global best
        if current_fitness < gbest_fitness:

            gbest = positions[i][:]
            gbest_fitness = current_fitness

# Display final result
h, l, t, b = gbest

cost = objective(gbest)
g, tau, sigma, delta, Pc = constraints(gbest)

print("\n----- OPTIMIZED WELDED BEAM DESIGN -----")

print("Weld thickness (h):", round(h, 6))
print("Weld length (l):", round(l, 6))
print("Beam height (t):", round(t, 6))
print("Beam thickness (b):", round(b, 6))

print("\nMinimum manufacturing cost:", round(cost, 6))

print("\n----- ENGINEERING RESULTS -----")
print("Shear stress:", round(tau, 4), "psi")
print("Bending stress:", round(sigma, 4), "psi")
print("Deflection:", round(delta, 6), "inches")
print("Buckling load:", round(Pc, 4), "lb")

# Check constraint satisfaction with a small numerical tolerance
if all(value <= 1e-5 for value in g):
```

```
        print("\nAll engineering constraints are satisfied.")
    else:
        print("\nSome engineering constraints are not satisfied.")


    print("Nischal AR")

# Run the program
pso()
```

output:

